Khmelnytskyi National University
Department of Information Technonogy

Report

Laboratory work №8

Discipline: “Programming”

Topic: “*Streams and files. Work with files*”

Completed: 1st year student, group SE-26-1 Nykon Hrytsyshyn
Name, Surname

Checked: Valeriy Martynyuk
Name, Surname

Khmelnytskyi, 2026

Purpose: to learn to use text files and work with them using the basic functions of the C language file system.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (matrix.h, files.h, tasks.h, utils.h, main.c, runner.c, matrix.c, files.c and utils.c), are provided in Appendices A…J.

In Table 1, general information about the tasks is provided.

Table 1 – Tasks overview

Task	What it does
1	Writes a sentence to a text file and all its letters in alphabetical order to the next line
2	Writes an n x m matrix to a text file, reads it back, prints the negative elements with their coordinates and adds them to the file
3	Adds a line S to the beginning of a text file

Task 1

Create a text file and write several sentences in it words, in the next line arrange alphabetically all the letters of this sentence. Save the result to a file and display it on the screen.

1.1. Block diagram of the program algorithm

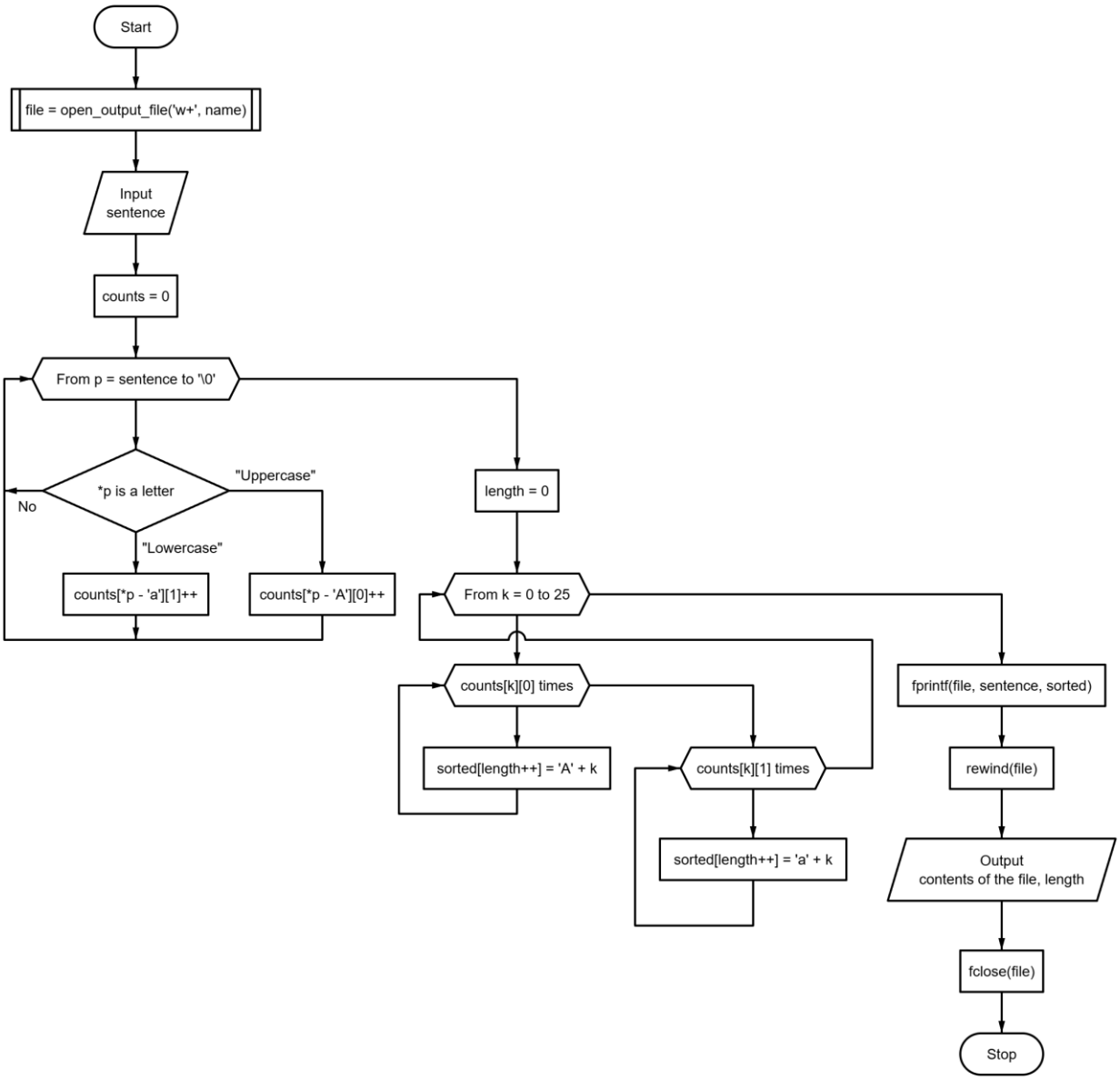


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "files.h"
#include "tasks.h"
#include "utils.h"

#include <ctype.h>
#include <stdio.h>

#define ALPHABET_SIZE 26

static size_t sort_letters(const char *const text, char *const
out) {
    size_t counts[ALPHABET_SIZE][2] = {{0}};
```

```
    for (const char *p = text; *p != '\0'; ++p) {
        const unsigned char c = (unsigned char)*p;

        if (isupper(c)) {
            ++counts[c - 'A'][0];
        } else if (islower(c)) {
            ++counts[c - 'a'][1];
        }
    }

    size_t length = 0;

    for (int k = 0; k < ALPHABET_SIZE; ++k) {
        for (size_t i = 0; i < counts[k][0]; ++i) {
            out[length++] = (char)('A' + k);
        }

        for (size_t i = 0; i < counts[k][1]; ++i) {
            out[length++] = (char)('a' + k);
        }
    }

    out[length] = '\0';

    return length;
}

void run_task1(void) {
    char name[MAX_LINE_SIZE];
    char text[MAX_LINE_SIZE];
    char sorted[MAX_LINE_SIZE];

    puts(
        "\n=== Task 1: a sentence and its letters in
alphabetical order ===\n"
    );

    FILE *const file = open_output_file("w+", name,
sizeof(name));

    if (file == NULL) {
        return;
    }

    if (!read_text_line("Enter the sentence: ", text,
sizeof(text))) {
        fclose(file);
        return;
```

```
    }

    const size_t count = sort_letters(text, sorted);

    if (fprintf(file, "%s\n%s\n", text, sorted) < 0 ||
fflush(file) != 0) {
        printf("Error: cannot write to \"%s\".\n", name);
        fclose(file);
        return;
    }

    rewind(file);

    printf("\nThe file \"%s\" contains:\n", name);
    print_stream(file);
    printf("\nNumber of letters: %zu\n", count);

    fclose(file);
}
```

1.3. Program results

```
=== Task 1: a sentence and its letters in alphabetical order ===

Enter the file name: task_1
Enter the sentence: I am a student of Khmelnytskyi National University

The file "task_1" contains:
I am a student of Khmelnytskyi National University
aaaadeeefhIiiiiKkllmmNnnnnoorssttttUuvyyy

Number of letters: 43
```

```
1   I am a student of Khmelnytskyi National University
2   aaaadeeefhIiiiiKkllmmNnnnnoorssttttUuvyyy
3
```

Fig. 2 – Task 1 results

Task 2

Create a text file and write a 6×7 matrix into it. Determine the coordinates of the negative elements. Display the coordinates together with the numbers on the screen and in the file.

2.1. Block diagram of the program algorithm

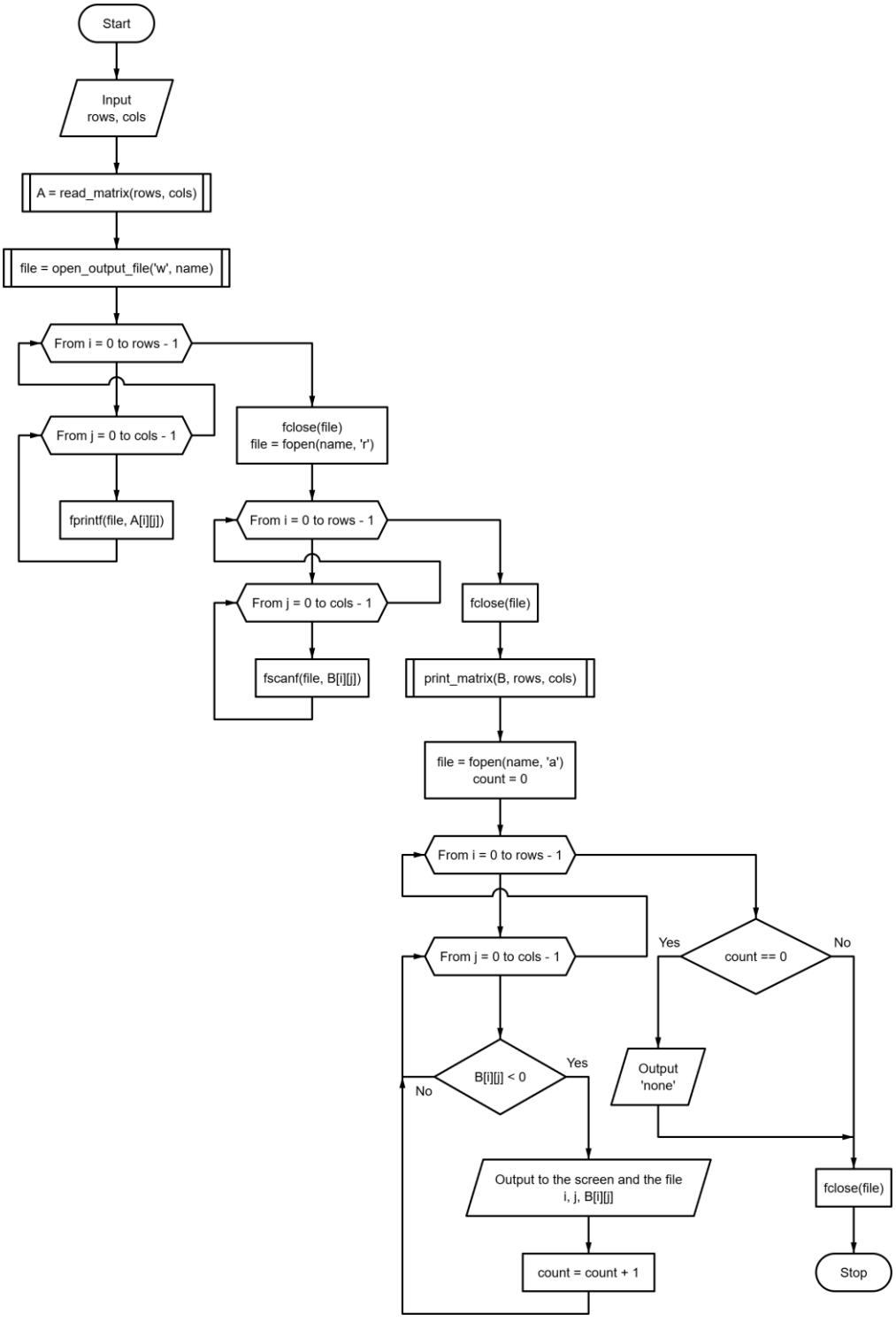


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#include "files.h"
#include "matrix.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static long print_negatives(
    FILE *const file,
    Matrix matrix,
    const long rows,
    const long cols
) {
    long count = 0;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            if (matrix[i][j] < 0) {
                printf("A[%ld][%ld] = %ld\n", i, j, matrix[i][j]);
                fprintf(file, "%ld, %ld: %ld\n", i, j,
matrix[i][j]);
                ++count;
            }
        }
    }

    return count;
}

void run_task2(void) {
    Matrix a;
    Matrix b;
    long rows = 0;
    long cols = 0;
    char name[MAX_LINE_SIZE];

    puts("\n=== Task 2: negative elements of a matrix in a file
===\n");

    if (!read_matrix_size("the matrix", &rows, &cols) ||
        !read_matrix("A", a, rows, cols)) {
        return;
    }

    FILE *const created = open_output_file("w", name,
sizeof(name));
```

```
    if (created == NULL) {
        return;
    }

    fclose(created);

    if (!write_matrix_file(name, a, rows, cols) ||
        !read_matrix_file(name, b, rows, cols)) {
        printf("Error: cannot write or read the file \"%s\".\n",
name);
        return;
    }

    printf("\nThe matrix is written to \"%s\" and read back:\n",
name);
    print_matrix("A", b, rows, cols);

    FILE *const file = fopen(name, "a");

    if (file == NULL) {
        printf("Error: cannot open \"%s\" for appending.\n",
name);
        return;
    }

    puts("\nNegative elements, A[row][column] = value:");
    fputs("\nNegative elements (row, column: value):\n", file);

    const long count = print_negatives(file, b, rows, cols);

    if (count == 0) {
        puts("none");
        fputs("none\n", file);
    }

    if (fclose(file) != 0) {
        printf("Error: cannot write to \"%s\".\n", name);
        return;
    }

    printf("\nNegative elements: %ld (they are added to the
file)\n", count);
}
```

2.3. Program results

```
=== Task 2: negative elements of a matrix in a file ===

Enter the number of rows of the matrix: 2
Enter the number of columns of the matrix: 3
Fill A with random numbers? (Y/n):
Enter the file name: task_2

The matrix is written to "task_2" and read back:
A (2 x 3) =
-23  22  49
-41   4  -8

Negative elements, A[row][column] = value:
A[0][0] = -23
A[1][0] = -41
A[1][2] = -8

Negative elements: 3 (they are added to the file)
```

```
1    -23 22 49
2    -41 4 -8
3
4    Negative elements (row, column: value):
5    0, 0: -23
6    1, 0: -41
7    1, 2: -8
```

Fig. 4 – Task 2 results

Task 3

Find the shortest and longest word in the given line, output these words and their length.

3.1. Block diagram of the program algorithm

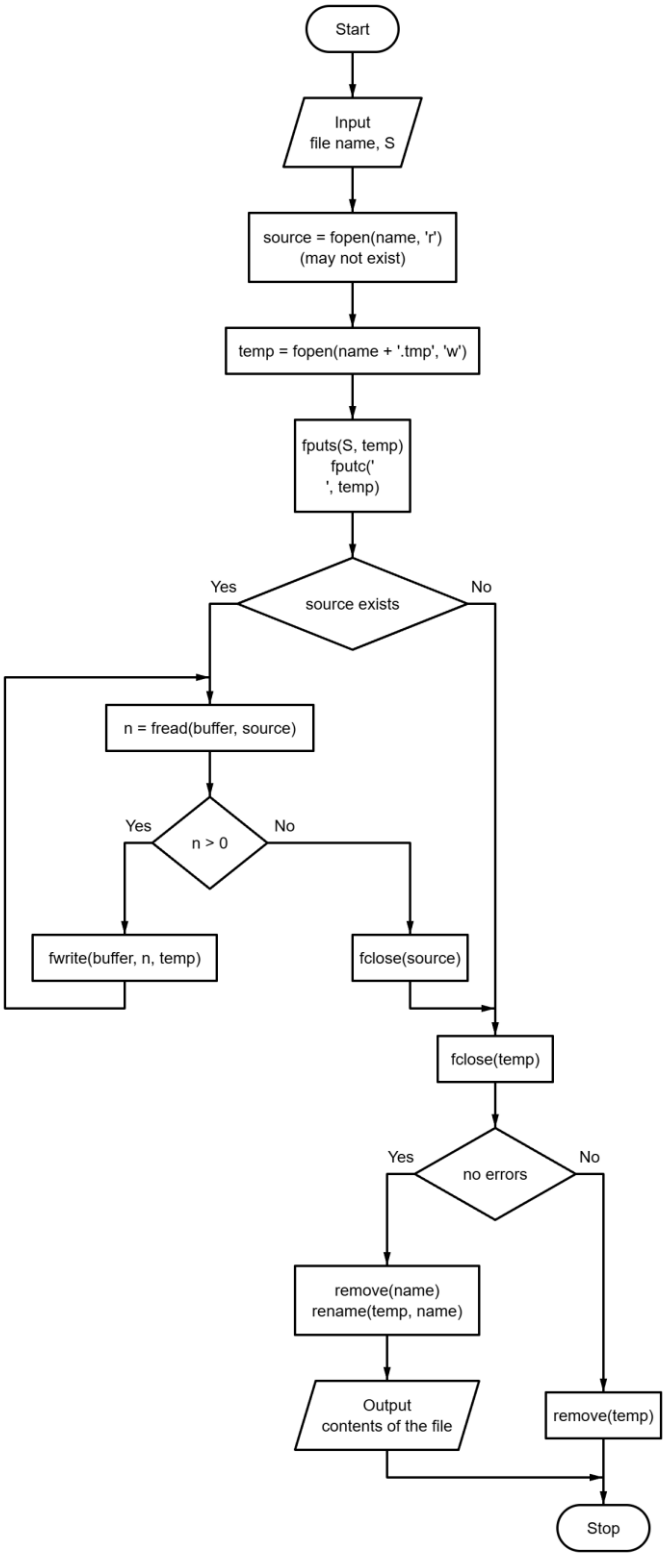


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#include "files.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

void run_task3(void) {
    char name[MAX_LINE_SIZE];
    char line[MAX_LINE_SIZE];

    puts("\n=== Task 3: add a line to the beginning of a file
===\n");
    puts("A file that does not exist is created.\n");

    if (!read_file_name(name, sizeof(name)) ||
        !read_text_line("Enter the line S: ", line, sizeof(line)))
{
        return;
    }

    if (!prepend_line(name, line)) {
        return;
    }

    printf("\nThe file \"%s\" now contains:\n", name);
    print_file(name);
}
```

3.3. Program results

```
=== Task 3: add a line to the beginning of a file ===

A file that does not exist is created.

Enter the file name: task_3
Enter the line S: NEW FIRST LINE
```

```
1    NEW FIRST LINE
2    Line 1
3    Line 2
4
```

Fig. 6 – Task 3 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in utilizing the C11 standard I/O file system streams and functions for text file manipulation. The tasks demonstrated effective functional decomposition across various file processing operations, including character sorting and stream rewinding using rewind(), formatted matrix state persistence and iterative file appending in "a" mode for negative element tracking, as well as safe file stream prepending using atomic temporary file buffer operations with fread(), fwrite(), remove(), and rename().

Furthermore, experience was gained in building cross-platform modular C software structured with CMake and Ninja, enforcing strict compiler diagnostics, and implementing robust input validation within an interactive task execution engine.

In Table 2, all results of the completed tasks are highlighted.

Table 2 – Results overview

Task	Input	Output
1	I am a student of Khmelnytskyi National University	file line 1: the sentence; line 2: aaaadeeefhIiiiiKkllmmNnnnnoorsssttttttUuvyyy; letters: 43
	123 !!	line 2 is empty; letters: 0
2	2x3 matrix 1 -2 3 / -4 5 -6	file: the matrix, then 0, 1: -2, 1, 0: -4, 1, 2: -6; negative elements: 3
3	file line one / line two, S = NEW FIRST LINE	file: NEW FIRST LINE, line one, line two
	a file that does not exist, S = only line	the file is created with the single line only line

APPENDIX A

Block diagram of the main program control flow algorithm

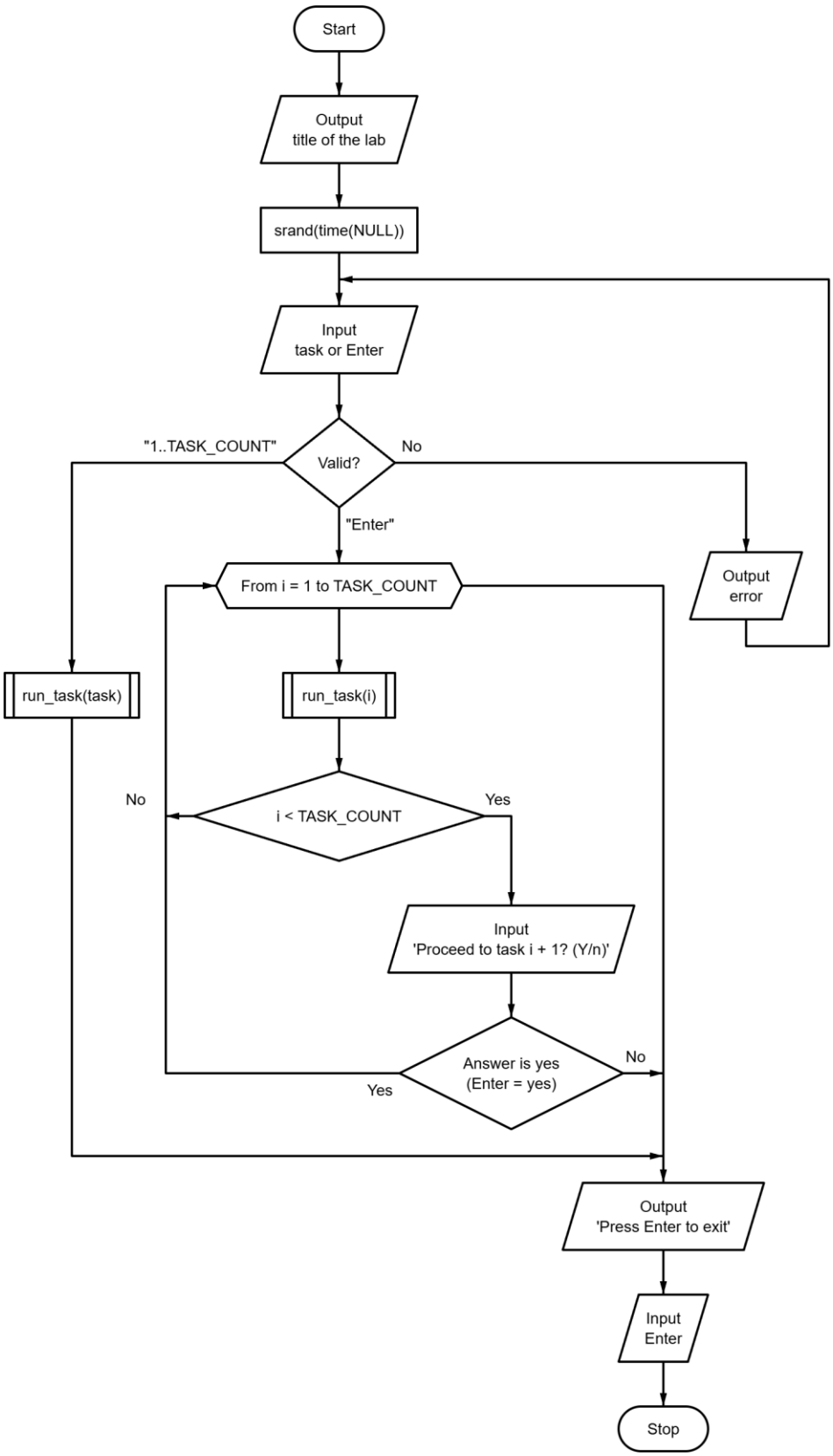


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

#define PROMPT_SIZE 96
#define MAX_LINE_SIZE 256

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);
bool read_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value
);
bool read_optional_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value,
    bool *out_provided
);

bool read_text_line(const char *prompt, char *buffer, size_t
size);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX D

Source code of the “files.h” file

```
#ifndef FILES_H
#define FILES_H

#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>

#define COPY_BUFFER_SIZE 1024
#define TEMP_SUFFIX ".tmp"

bool read_file_name(char *name, size_t size);
FILE *open_output_file(const char *mode, char *name, size_t size);

void print_stream(FILE *file);
bool print_file(const char *name);

bool prepend_line(const char *name, const char *line);

#endif // FILES_H
```

APPENDIX E

Source code of the “matrix.h” file

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stdbool.h>

#define MAX_MATRIX_DIM 15
#define MAX_ELEMENT_ABS 1000L
#define RANDOM_ABS 50

typedef long Matrix[MAX_MATRIX_DIM][MAX_MATRIX_DIM];

bool read_matrix_size(const char *name, long *out_rows, long
*out_cols);
bool read_matrix(const char *name, Matrix matrix, long rows, long
cols);
void print_matrix(const char *name, Matrix matrix, long rows, long
cols);

bool write_matrix_file(const char *name, Matrix matrix, long rows,
long cols);
bool read_matrix_file(const char *name, Matrix matrix, long rows,
long cols);

#endif // MATRIX_H
```

APPENDIX F

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    puts(
        "=================================================\n"
        " Laboratory work 8: Streams and files (variant 9) \n"
        "=================================================\n"
    );

    srand((unsigned)time(NULL)); // NOLINT(*-msc51-cpp)

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt,
        sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks
[ALL]: ",
        TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task,
&provided)) {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX G

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d?
(Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX H

Source code of the “files.c” file

```
#include "files.h"

#include "utils.h"

#include <errno.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>

bool read_file_name(char *const name, const size_t size) {
    while (true) {
        if (!read_text_line("Enter the file name: ", name, size))
{
            return false;
        }

        if (name[0] != '\0') {
            return true;
        }

        puts("Error: the file name must not be empty.");
    }
}

FILE *open_output_file(
    const char *const mode,
    char *const name,
    const size_t size
) {
    while (true) {
        if (!read_file_name(name, size)) {
            return NULL;
        }

        FILE *const existing = fopen(name, "r");

        if (existing != NULL) {
            fclose(existing);

            if (!confirm(
                    "The file exists and will be overwritten.
Continue? "
                    "(y/N): ",
                    false
```

```
                )) {
                if (feof(stdin)) {
                    return NULL;
                }

                continue;
            }
        }

        FILE *const file = fopen(name, mode);

        if (file != NULL) {
            return file;
        }

        printf("Error: cannot open \"%s\": %s.\n", name,
strerror(errno));
    }
}

void print_stream(FILE *const file) {
    char line[MAX_LINE_SIZE];

    while (fgets(line, sizeof(line), file) != NULL) {
        fputs(line, stdout);
    }
}

bool print_file(const char *const name) {
    FILE *const file = fopen(name, "r");

    if (file == NULL) {
        printf("Error: cannot open \"%s\": %s.\n", name,
strerror(errno));
        return false;
    }

    print_stream(file);
    fclose(file);

    return true;
}

static bool copy_stream(FILE *const from, FILE *const to) {
    char buffer[COPY_BUFFER_SIZE];
    size_t count;
```

```
    while ((count = fread(buffer, 1, sizeof(buffer), from)) > 0) {
        if (fwrite(buffer, 1, count, to) != count) {
            return false;
        }
    }

    return !ferror(from);
}

static bool replace_file(
    const char *const temp_name,
    const char *const name,
    const bool existed
) {
    if (existed && remove(name) != 0) {
        printf("Error: cannot replace \"%s\": %s.\n", name,
strerror(errno));
        remove(temp_name);
        return false;
    }

    if (rename(temp_name, name) != 0) {
        printf(
            "Error: cannot rename \"%s\" to \"%s\": %s.\n",
            temp_name,
            name,
            strerror(errno)
        );
        return false;
    }

    return true;
}

bool prepend_line(const char *const name, const char *const line)
{
    char temp_name[MAX_LINE_SIZE + sizeof(TEMP_SUFFIX)];

    snprintf(temp_name, sizeof(temp_name), "%s%s", name,
TEMP_SUFFIX);

    FILE *const source = fopen(name, "r");
    const bool existed = source != NULL;

    if (!existed && errno != ENOENT) {
        printf("Error: cannot open \"%s\": %s.\n", name,
strerror(errno));
```

```
        return false;
    }

    FILE *const temp = fopen(temp_name, "w");

    if (temp == NULL) {
        printf(
            "Error: cannot create \"%s\": %s.\n", temp_name,
strerror(errno)
        );

        if (existed) {
            fclose(source);
        }

        return false;
    }

    bool ok = fputs(line, temp) >= 0 && fputc('\n', temp) != EOF;

    if (existed) {
        ok = ok && copy_stream(source, temp);
        fclose(source);
    }

    if (fclose(temp) != 0) {
        ok = false;
    }

    if (!ok) {
        printf("Error: cannot write \"%s\".\n", temp_name);
        remove(temp_name);
        return false;
    }

    return replace_file(temp_name, name, existed);
}
```

APPENDIX I

Source code of the “matrix.c” file

```
#include "matrix.h"

#include "utils.h"

#include <errno.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

bool read_matrix_size(
    const char *const name,
    long *const out_rows,
    long *const out_cols
) {
    char prompt[PROMPT_SIZE];

    snprintf(prompt, sizeof(prompt), "Enter the number of rows of
%s: ", name);

    if (!read_long_in_range(prompt, 1, MAX_MATRIX_DIM, out_rows))
{
        return false;
    }

    snprintf(
        prompt, sizeof(prompt), "Enter the number of columns of
%s: ", name
    );

    return read_long_in_range(prompt, 1, MAX_MATRIX_DIM,
out_cols);
}

bool read_matrix(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    char prompt[PROMPT_SIZE];

    snprintf(
        prompt, sizeof(prompt), "Fill %s with random numbers?
```

```
(Y/n): ", name
    );

    const bool random_fill = confirm(prompt, true);

    if (feof(stdin)) {
        return false;
    }

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            if (random_fill) {
                const int r = rand(); // NOLINT(*-msc50-cpp)
                matrix[i][j] = r % (2 * RANDOM_ABS + 1) -
RANDOM_ABS;
                continue;
            }

            snprintf(prompt, sizeof(prompt), "%s[%ld][%ld] = ",
name, i, j);

            if (!read_long_in_range(
                    prompt, -MAX_ELEMENT_ABS, MAX_ELEMENT_ABS,
&matrix[i][j]
                )) {
                return false;
            }
        }
    }

    return true;
}

void print_matrix(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    char text[32];
    size_t width = 1;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            snprintf(text, sizeof(text), "%ld", matrix[i][j]);

            if (strlen(text) > width) {
```

```
                width = strlen(text);
            }
        }
    }

    printf("%s (%ld x %ld) =\n", name, rows, cols);

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            printf(j == 0 ? "%*ld" : " %*ld", (int)width,
matrix[i][j]);
        }

        puts("");
    }
}

bool write_matrix_file(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    FILE *const file = fopen(name, "w");

    if (file == NULL) {
        return false;
    }

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            fprintf(file, j == 0 ? "%ld" : " %ld", matrix[i][j]);
        }

        fputc('\n', file);
    }

    return fclose(file) == 0;
}

static bool parse_row(
    const char *const line,
    Matrix matrix,
    const long row,
    const long cols
) {
    const char *position = line;
```

```
    for (long j = 0; j < cols; ++j) {
        char *end;

        errno = 0;
        const long value = strtol(position, &end, 10);

        if (end == position || errno != 0) {
            return false;
        }

        matrix[row][j] = value;
        position = end;
    }

    return true;
}

bool read_matrix_file(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    FILE *const file = fopen(name, "r");

    if (file == NULL) {
        return false;
    }

    char line[MAX_LINE_SIZE];
    bool ok = true;

    for (long i = 0; i < rows && ok; ++i) {
        ok = fgets(line, sizeof(line), file) != NULL &&
             parse_row(line, matrix, i, cols);
    }

    fclose(file);

    return ok;
}
```

APPENDIX J

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr,
void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }

        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr,
void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void
*out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
```

```
    const size_t count,
    void *const outs[],
    const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];
```

```
    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, double *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {
    while (true) {
        if (read_long(prompt, out_value)) {
            if (*out_value >= min && *out_value <= max) {
```

```
                return true;
            }

            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter an integer.");
        }
    }
}

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press
Enter.");
        } else if (value < min || value > max) {
            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else {
            *out_value = value;
            *out_provided = true;
```

```
            return true;
        }
    }
}

bool read_text_line(
    const char *const prompt,
    char *const buffer,
    const size_t size
) {
    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, (int)size, stdin) == NULL) {
            return false;
        }

        char *const newline = strchr(buffer, '\n');

        if (newline != NULL) {
            *newline = '\0';
            return true;
        }

        if (feof(stdin)) {
            return true;
        }

        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;

        printf("Error: the line must be shorter than %zu
characters.\n", size);
    }
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
```

```
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL &&
!feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }

            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len -
1])) {
                len--;
            }

            if (len == 0) {
                return default_answer;
            }

            if (equals_ignore_case(start, len, "y") ||
                equals_ignore_case(start, len, "yes")) {
                return true;
            }
```

```
            if (equals_ignore_case(start, len, "n") ||
                equals_ignore_case(start, len, "no")) {
                return false;
            }
        }

        printf("Please answer 'y' or 'n'.\n");
    }
}
```